Programming Languages Concepts

LAB #3 / PART 2

Basic Operators in Programming Language

Chosen implementation language: Ruby

Student name: Igors

Student surname: Oļeiņikovs

St. code: 93642

Group: 4501BTA

Lecturer: Karina Kostjkina

RIGA

2026

Content

1. Purpose and task
2. Program code and results
3. Conclusions
4. Literature

Purpose and Task

The purpose of this laboratory work is to practise arithmetic operators, transformations, selection statements, loops, constants, input validation, and simple console interaction in a programming language. Ruby was selected for the implementation because its syntax is concise and expressive, which makes the small console programs easy to read and compare with other languages.

Implemented tasks:

1. Find the roots of a quadratic equation from coefficients `a`, `b`, and `c`.
2. Multiply two real numbers and output a rounded or truncated result.
3. Convert temperature either from Celsius to Fahrenheit or from Fahrenheit to Celsius.
4. Divide two integers and display the integer quotient with the remainder.
5. Calculate the average of three real numbers to two decimal places.
6. Calculate the area and perimeter of a rectangle.
7. Check whether the user is old enough to apply for a driving license.
8. Use a loop to sum whole numbers until the user enters `0`.
9. Check whether a password is correct and discuss secure password handling.
10. Calculate the area and circumference of a circle using the constant `pi = 3.14159`.

Program Code and Results

Task 1. Quadratic equation roots

The program reads the coefficients of the quadratic equation, calculates the discriminant, and then outputs two real roots, one repeated root, or a message that no real roots exist.

```
#!/usr/bin/env ruby

def read_float(prompt)
  loop do
    print prompt
    input = gets&.strip
    exit if input.nil?

    value = Float(input, exception: false)
    return value unless value.nil?

    puts 'Please enter a valid number.'
  end
end

a = read_float('Enter coefficient a: ')
b = read_float('Enter coefficient b: ')
c = read_float('Enter coefficient c: ')

if a.zero?
  puts 'Coefficient a must not be zero for a quadratic equation.'
  exit
end

discriminant = (b * b) - (4 * a * c)

puts format('Discriminant: %.3f', discriminant)

if discriminant.negative?
  puts 'The equation has no real roots.'
elsif discriminant.zero?
  root = -b / (2.0 * a)
  puts format('x1 = x2 = %.3f', root)
else
  sqrt_discriminant = Math.sqrt(discriminant)
  first_root = (-b + sqrt_discriminant) / (2.0 * a)
  second_root = (-b - sqrt_discriminant) / (2.0 * a)

  puts format('x1 = %.3f', first_root)
  puts format('x2 = %.3f', second_root)
end
```

Task 1 Result

task1_quadratic_roots.rb

```
$ ruby task1_quadratic_roots.rb
Enter coefficient a: 1
Enter coefficient b: -3
Enter coefficient c: 2
Discriminant: 1.000
x1 = 2.000
x2 = 1.000
```

Figure 1. Result of Task 1

Task 2. Product with rounding or truncation

The program multiplies two real numbers and applies the user-selected output mode. In this implementation, the result is either rounded or truncated to a whole number, which clearly demonstrates the corresponding Ruby operations.

```ruby
#!/usr/bin/env ruby

def read_float(prompt)
  loop do
    print prompt
    input = gets&.strip
    exit if input.nil?

    value = Float(input, exception: false)
    return value unless value.nil?

    puts 'Please enter a valid number.'
  end
end

def read_result_mode
  loop do
    print "Choose result type ('rounded' or 'truncated'): "
    input = gets&.strip&.downcase
    exit if input.nil?

    return input if %w[rounded truncated].include?(input)

    puts "Please type 'rounded' or 'truncated'."
  end
end

first_number = read_float('Enter the first real number: ')
second_number = read_float('Enter the second real number: ')
result_mode = read_result_mode

product = first_number * second_number

puts format('Exact product: %.4f', product)

if result_mode == 'rounded'
  puts "Rounded product: #{product.round}"
else
  puts "Truncated product: #{product.truncate}"
end
```

Task 2 Result

task2_product_round_or_truncate.rb

```
$ ruby task2_product_round_or_truncate.rb
Enter the first real number: 12.75
Enter the second real number: 3.2
Choose result type ('rounded' or 'truncated'): rounded
Exact product: 40.8000
Rounded product: 41
```

Figure 2. Result of Task 2

Task 3. Temperature conversion

The program demonstrates transformation formulas and a conditional choice between Celsius-to-Fahrenheit and Fahrenheit-to-Celsius conversion.

```
#!/usr/bin/env ruby

def read_float(prompt)
  loop do
    print prompt
    input = gets&.strip
    exit if input.nil?

    value = Float(input, exception: false)
    return value unless value.nil?

    puts 'Please enter a valid number.'
  end
end

def read_conversion_choice
  loop do
    print "Convert from C to F or from F to C? Enter 'C' or 'F': "
    input = gets&.strip&.upcase
    exit if input.nil?

    return input if %w[C F].include?(input)

    puts "Please enter 'C' or 'F'."
  end
end

temperature = read_float('Enter the temperature value: ')
choice = read_conversion_choice

if choice == 'C'
  converted_temperature = (temperature * 9.0 / 5.0) + 32
  puts format('%.2f C = %.2f F', temperature, converted_temperature)
else
  converted_temperature = (temperature - 32) * 5.0 / 9.0
  puts format('%.2f F = %.2f C', temperature, converted_temperature)
end
```

Task 3 Result

task3_temperature_converter.rb

```
$ ruby task3_temperature_converter.rb
Enter the temperature value: 25
Convert from C to F or from F to C? Enter 'C' or 'F': C
25.00 C = 77.00 F
```

Figure 3. Result of Task 3

Program Code and Results (continued)

Task 4. Integer division with remainder

The program accepts two integers and uses Ruby's integer division support to output the quotient and remainder in the required form.

```
#!/usr/bin/env ruby

def read_integer(prompt)
  loop do
    print prompt
    input = gets&.strip
    exit if input.nil?

    value = Integer(input, exception: false)
    return value unless value.nil?

    puts 'Please enter a valid integer.'
  end
end

dividend = read_integer('Enter the first integer: ')
divisor = read_integer('Enter the second integer: ')

if divisor.zero?
  puts 'Division by zero is not allowed.'
  exit
end

quotient, remainder = dividend.divmod(divisor)

puts "#{dividend}/#{divisor} = #{quotient} remainder #{remainder}"
```

Task 4 Result

task4_integer_division_with_remainder.rb

```
$ ruby task4_integer_division_with_remainder.rb
Enter the first integer: 11
Enter the second integer: 3
11/3 = 3 remainder 2
```

Figure 4. Result of Task 4

Task 5. Average of three real numbers

The program reads three real numbers, calculates their arithmetic mean, and formats the output to two decimal places.

```
#!/usr/bin/env ruby

def read_float(prompt)
  loop do
    print prompt
    input = gets&.strip
    exit if input.nil?

    value = Float(input, exception: false)
    return value unless value.nil?

    puts 'Please enter a valid number.'
  end
end

first_number = read_float('Enter the first real number: ')
second_number = read_float('Enter the second real number: ')
third_number = read_float('Enter the third real number: ')

average = (first_number + second_number + third_number) / 3.0

puts format('Average: %.2f', average)
```

Task 5 Result

task5_average_of_three_numbers.rb

```
$ ruby task5_average_of_three_numbers.rb
Enter the first real number: 4.5
Enter the second real number: 7.2
Enter the third real number: 9.0
Average: 6.90
```

Figure 5. Result of Task 5

Task 6. Rectangle area and perimeter

The program uses non-negative input validation and then calculates the rectangle area and perimeter with simple arithmetic operators.

```ruby
#!/usr/bin/env ruby

def read_float(prompt)
  loop do
    print prompt
    input = gets&.strip
    exit if input.nil?

    value = Float(input, exception: false)
    return value if !value.nil? && value >= 0

    puts 'Please enter a valid non-negative number.'
  end
end

length = read_float('Enter the rectangle length: ')
breadth = read_float('Enter the rectangle breadth: ')

area = length * breadth
perimeter = 2 * (length + breadth)

puts format('Area: %.2f', area)
puts format('Perimeter: %.2f', perimeter)
```

Task 6 Result

task6_rectangle_area_and_perimeter.rb

```
$ ruby task6_rectangle_area_and_perimeter.rb
Enter the rectangle length: 8.5
Enter the rectangle breadth: 4
Area: 34.00
Perimeter: 25.00
```

Figure 6. Result of Task 6

Program Code and Results (continued)

Task 7. Driving license eligibility

The program checks whether the user has reached the legal age of 18. If not, it calculates the remaining number of years.

```
#!/usr/bin/env ruby

def read_integer(prompt)
  loop do
    print prompt
    input = gets&.strip
    exit if input.nil?

    value = Integer(input, exception: false)
    return value if !value.nil? && value >= 0

    puts 'Please enter a valid non-negative integer.'
  end
end

age = read_integer('Enter your age: ')

if age >= 18
  puts 'You are of legal age to apply for a driving license.'
else
  years_remaining = 18 - age
  puts "You can apply for a driving license in #{years_remaining}
year(s)."
end
```

Task 7 Result

task7_driving_license_check.rb

```
$ ruby task7_driving_license_check.rb
Enter your age: 16
You can apply for a driving license in 2 year(s).
```

Figure 7. Result of Task 7

Task 8. Sum of whole numbers until zero

The program demonstrates a loop. It repeatedly reads whole numbers, adds them to the total, and stops when the user enters zero.

```ruby
#!/usr/bin/env ruby

sum = 0

loop do
  print 'Enter a whole number (0 to stop): '
  input = gets&.strip
  exit if input.nil?

  value = Integer(input, exception: false)

  if value.nil? || value.negative?
    puts 'Please enter a valid whole number.'
    next
  end

  break if value.zero?

  sum += value
end

puts "Sum: #{sum}"
```

Task 8 Result

task8_sum_until_zero.rb

```
$ ruby task8_sum_until_zero.rb
Enter a whole number (0 to stop): 5
Enter a whole number (0 to stop): 7
Enter a whole number (0 to stop): 3
Enter a whole number (0 to stop): 0
Sum: 15
```

Figure 8. Result of Task 8

Program Code and Results (continued)

Task 9. Password check

The program checks a password and prints whether it is correct. To answer the security question from the assignment, the solution does not store the password in plain text. Instead, it stores a salted SHA-256 hash and compares it with the hash of the entered value. The program also hides the password when it is typed in a real terminal.

```
#!/usr/bin/env ruby

require 'digest'
require 'io/console'

PASSWORD_SALT = 'lab3-ruby-salt'
PASSWORD_HASH =
'faeb6e294eae93f9ca188c6646122fab2b2b2d8bbc9f1078e4a40cb9feb2301a'

print 'Enter password: '
if STDIN.tty?
  password = STDIN.noecho(&:gets)
  puts
else
  password = gets
end

exit if password.nil?

entered_password = password.strip
entered_hash = Digest::SHA256.hexdigest("#{PASSWORD_SALT}#
{entered_password}")

if entered_hash == PASSWORD_HASH
  puts 'Password is correct.'
else
  puts 'Password is incorrect.'
end

puts 'Security note: passwords should be stored as salted hashes and
entered without displaying them on screen.'
```

Task 9 Result

task9_password_check.rb

```
$ ruby task9_password_check.rb
Enter password:
Password is correct.
Security note: passwords should be stored as salted hashes and entered without
displaying them on screen.
```

Figure 9. Result of Task 9

Task 10. Circle area and circumference

The program uses the constant value `pi = 3.14159` and calculates both the area and the circumference of a circle for a non-negative radius.

```
#!/usr/bin/env ruby

PI_VALUE = 3.14159

def read_float(prompt)
  loop do
    print prompt
    input = gets&.strip
    exit if input.nil?

    value = Float(input, exception: false)
    return value if !value.nil? && value >= 0

    puts 'Please enter a valid non-negative number.'
  end
end

radius = read_float('Enter the radius of the circle: ')

area = PI_VALUE * radius * radius
circumference = 2 * PI_VALUE * radius

puts format('Area: %.4f', area)
puts format('Circumference: %.4f', circumference)
```

Task 10 Result

task10_circle_area_and_circumference.rb

```
$ ruby task10_circle_area_and_circumference.rb
Enter the radius of the circle: 5
Area: 78.5398
Circumference: 31.4159
```

Figure 10. Result of Task 10

Conclusions

In this laboratory work, ten console programs were implemented in Ruby to demonstrate arithmetic operations, transformation formulas, conditions, loops, constants, and input handling. Ruby made the programs concise and readable, while still providing the required numerical operations and standard-library tools.

Compared with C++, Ruby requires much less boilerplate and no manual type declarations, which speeds up the development of small console tasks. Compared with Java, Ruby is again shorter and faster to write, although Java provides stronger compile-time type checking. Compared with Python, Ruby offers a similar level of productivity; both languages are suitable for educational console tasks, but Ruby's block-oriented syntax and built-in methods make the solutions especially compact.

The password task also showed that Ruby can support better security practices even in a simple laboratory program: passwords should be stored as salted hashes rather than plain text, and user input should not be displayed while typing.

Literature

1. Basic Language Operators. Lab lessons 3-4. Course assignment PDF.
2. Ruby Documentation. Core classes `Integer`, `Float`, and `Math`.
3. Ruby Documentation. `io/console` standard library.
4. Ruby Documentation. `Digest` standard library.